Insurance Claim Triage: Dashboard Description

Nov 3, 2026 · @Gaurav Chavan

1. Overview

The dashboard is a Streamlit app where a claims reviewer works through a ranked queue, reads why each claim scored as it did, records a decision and can teach the system a lesson. Customers appear by ID only, and a green band is never an approval.

Item	Detail
Who uses it	A claims reviewer (daily work), a team lead (reads the Evaluation page to see how far to trust the system) and the project team
What it does	Ranks claims least trusted first, explains each score, records every decision in an audit trail, and passes reviewer lessons through a safety check
What it never does	Pay, approve or reject a claim, change a score, or open the privacy vault or the evaluation answer keys
Pages	Review queue, Claim, Knowledge base, Evaluation
Sidebar	Page navigation, the reviewer's name, and which assessment to show (rules-only or a saved agent run)
Technology	Streamlit 1.66 or later, pandas, DuckDB (read-only for data, read-write for the app database)

Start it

```
uv run python -m claim_triage.app.backfill
uv run streamlit run src/claim_triage/app/dashboard.py
```

The first command is free: it runs the rules-only system for every claim and stores any saved agent runs. The second opens the app, by default at `http://localhost:8501`. Setting `CLAIM_TRIAGE_APP_DB` to another file points the app at a scratch copy of its database.

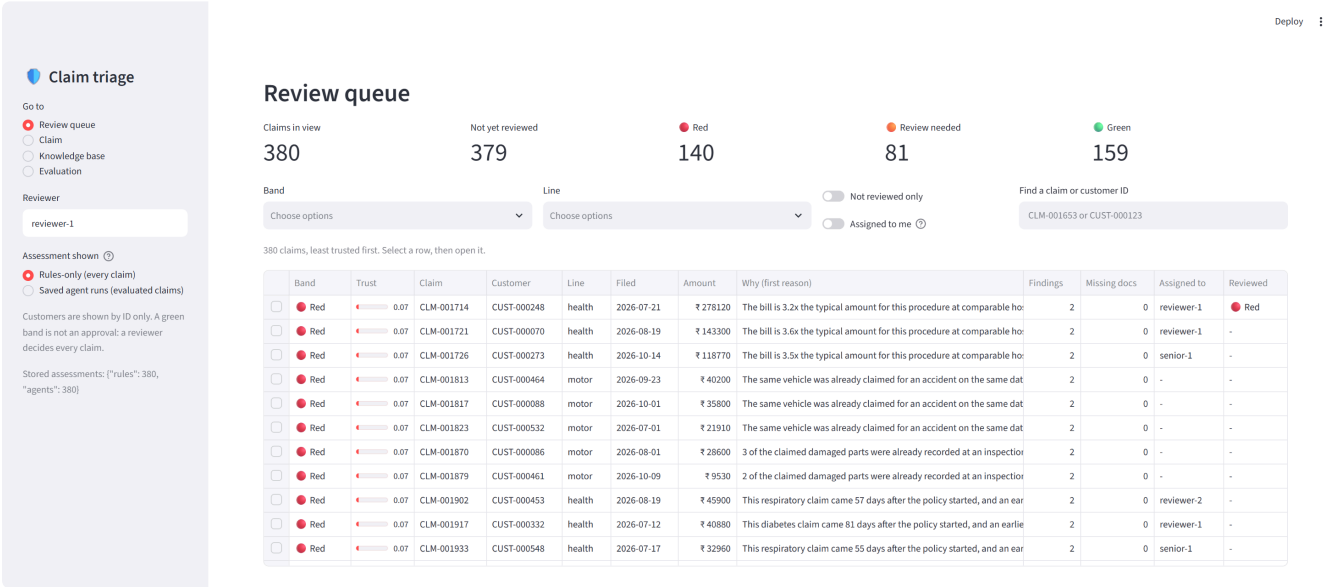
About the screenshots. Every picture was captured automatically from the running app in a headless Edge browser, on a scratch copy of the app database, so no real decision was touched. All customers, claims and documents are synthetic. Orange numbered frames mark the components that the tables describe.

2. The four pages at a glance

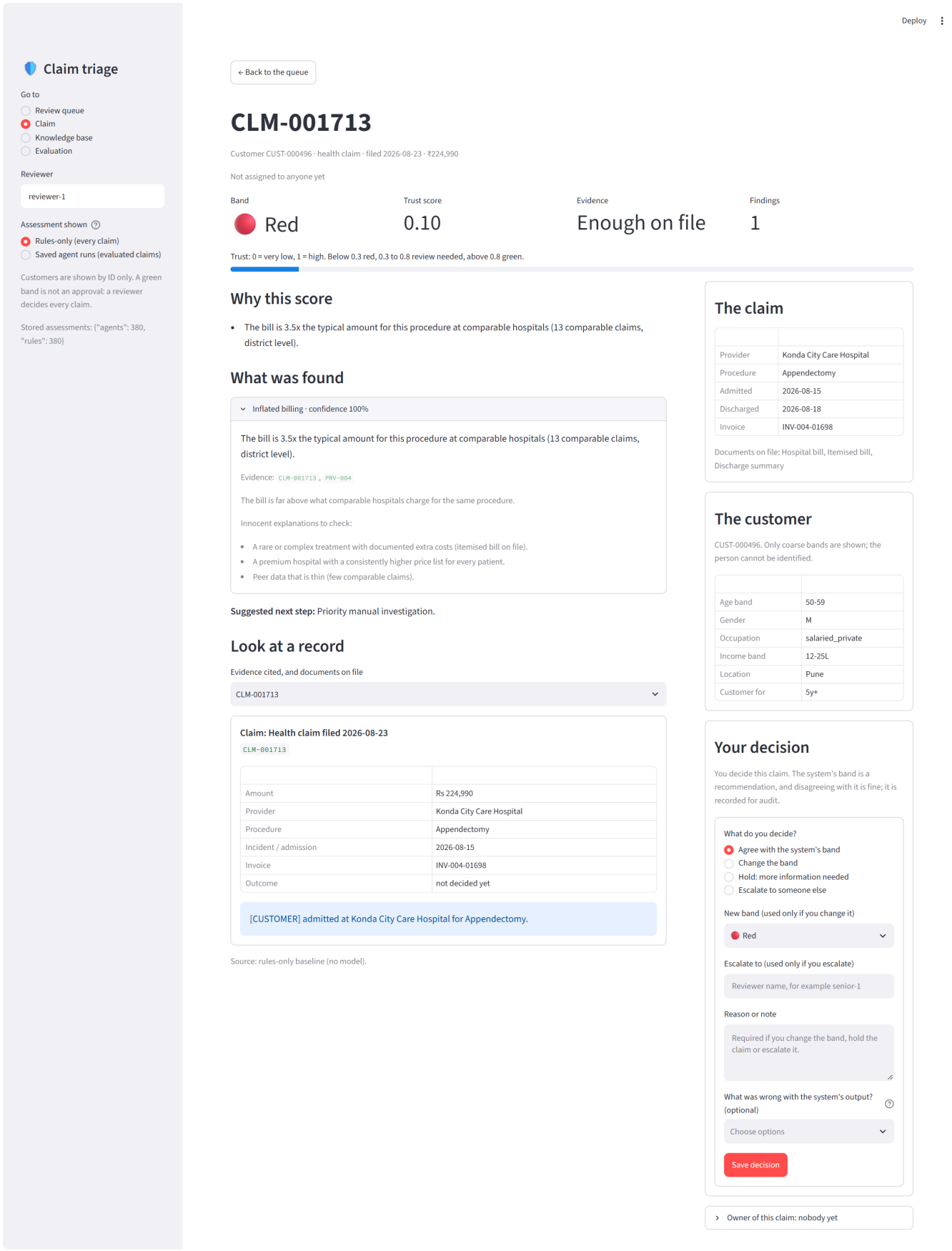
Each page has one job, and the sidebar on the left is the same on all of them.

Page	Its job	Main data
Review queue	Rank all claims, filter them and pick one to open	stored assessments plus `silver.duckdb`
Claim	Explain one claim's score, show evidence and record the reviewer's decision	one assessment, the claim and customer views, `decisions`
Knowledge base	Offer findings from decided claims as lessons and show what happened to them	`decisions`, `signals`, the safety check
Evaluation	Show every test run honestly, with what each does and does not prove	`eval/*_cases.csv` and the signal counts

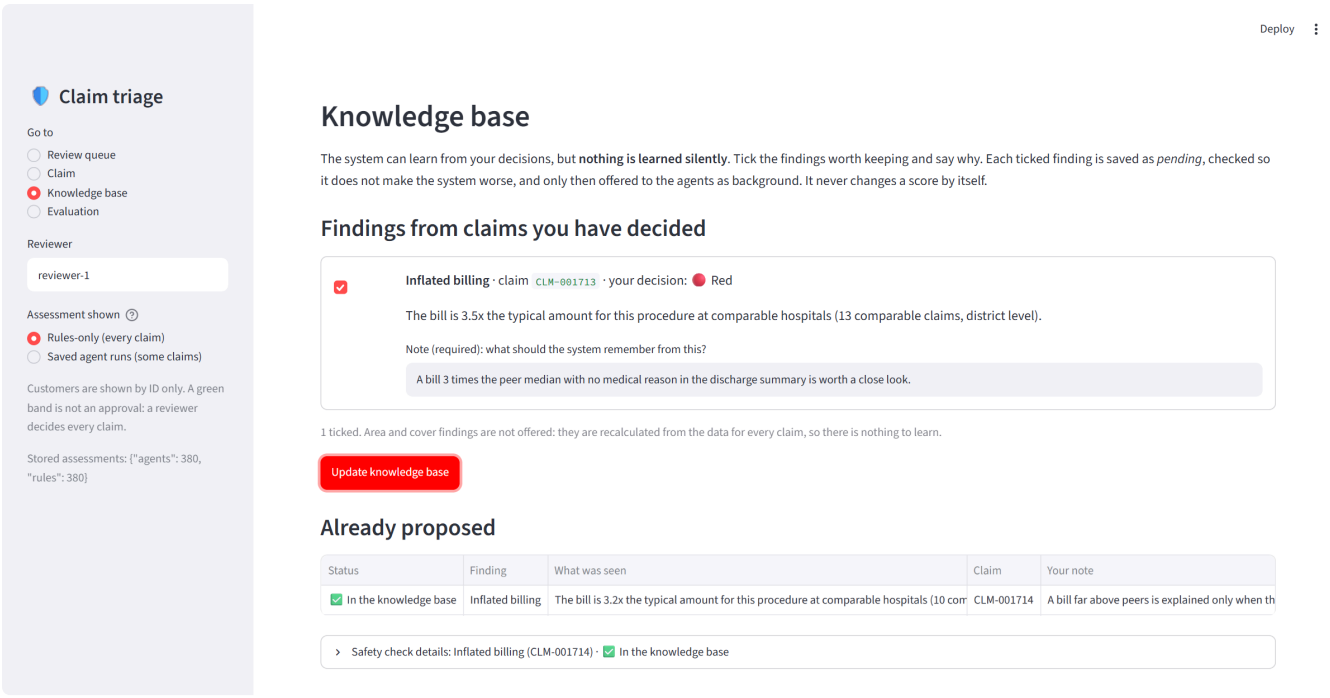
Review queue



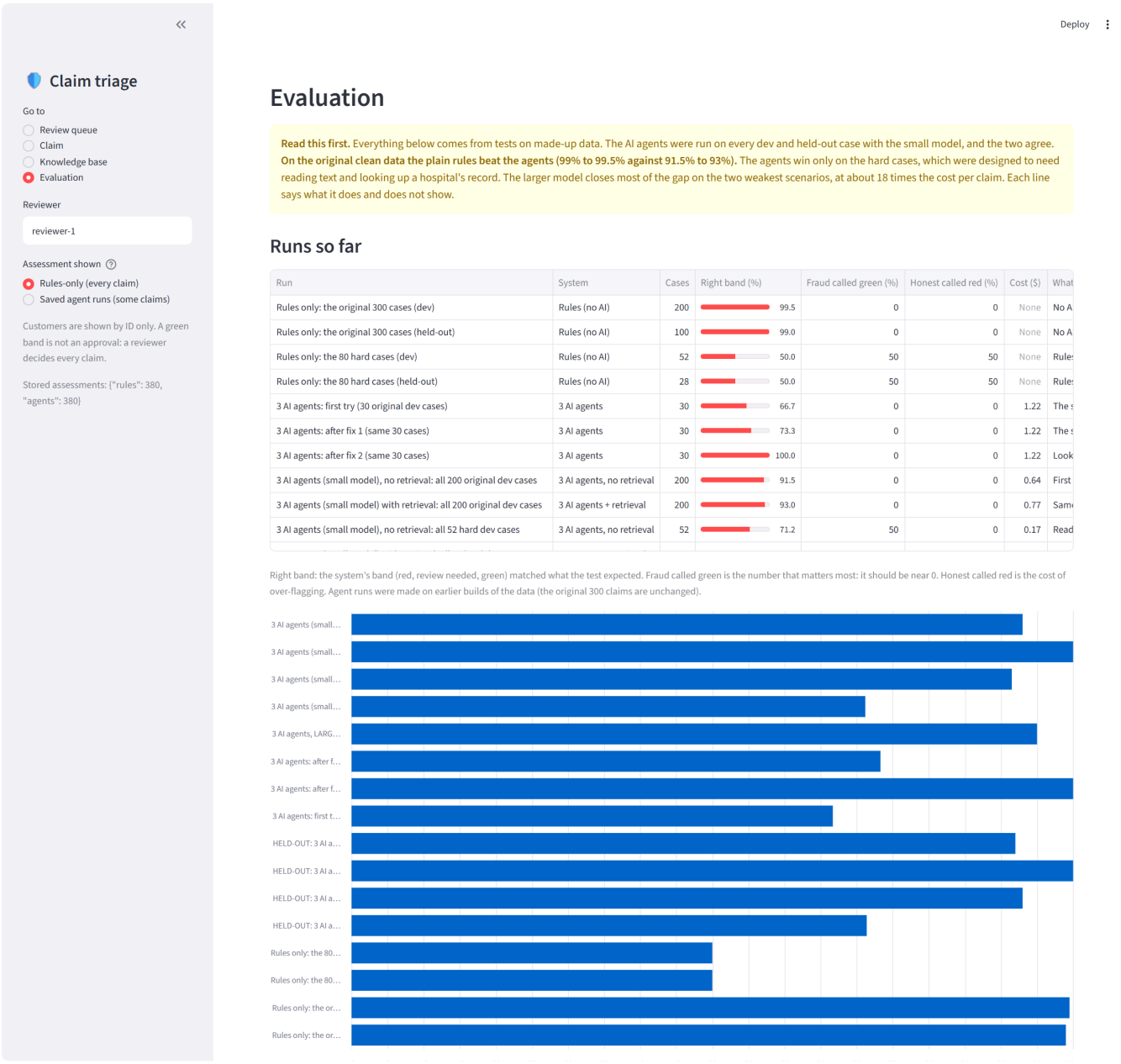
Claim



Knowledge base


Deploy
Claim triage
Go to
Review queue
Claim
Knowledge base
Evaluation
Reviewer
reviewer-1
Assessment shown
Rules-only (every claim)
Saved agent runs (some claims)
Customers are shown by ID only. A green band is not an approval: a reviewer decides every claim.
Stored assessments: {"agents": 380, "rules": 380}
Knowledge base
The system can learn from your decisions, but nothing is learned silently. Tick the findings worth keeping and say why. Each ticked finding is saved as pending, checked so it does not make the system worse, and only then offered to the agents as background. It never changes a score by itself.
Findings from claims you have decided
Inflated billing · claim CLM-001713 · your decision: Red
The bill is 3.5x the typical amount for this procedure at comparable hospitals (13 comparable claims, district level).
Note (required): what should the system remember from this?
A bill 3 times the peer median with no medical reason in the discharge summary is worth a close look.
1 ticked. Area and cover findings are not offered: they are recalculated from the data for every claim, so there is nothing to learn.
Update knowledge base
Already proposed
Status
Finding
What was seen
Claim
Your note
In the knowledge base
Inflated billing
The bill is 3.2x the typical amount for this procedure at comparable hospitals (10 com
CLM-001714
A bill far above peers is explained only when th
Safety check details: Inflated billing (CLM-001714) · In the knowledge base

Evaluation



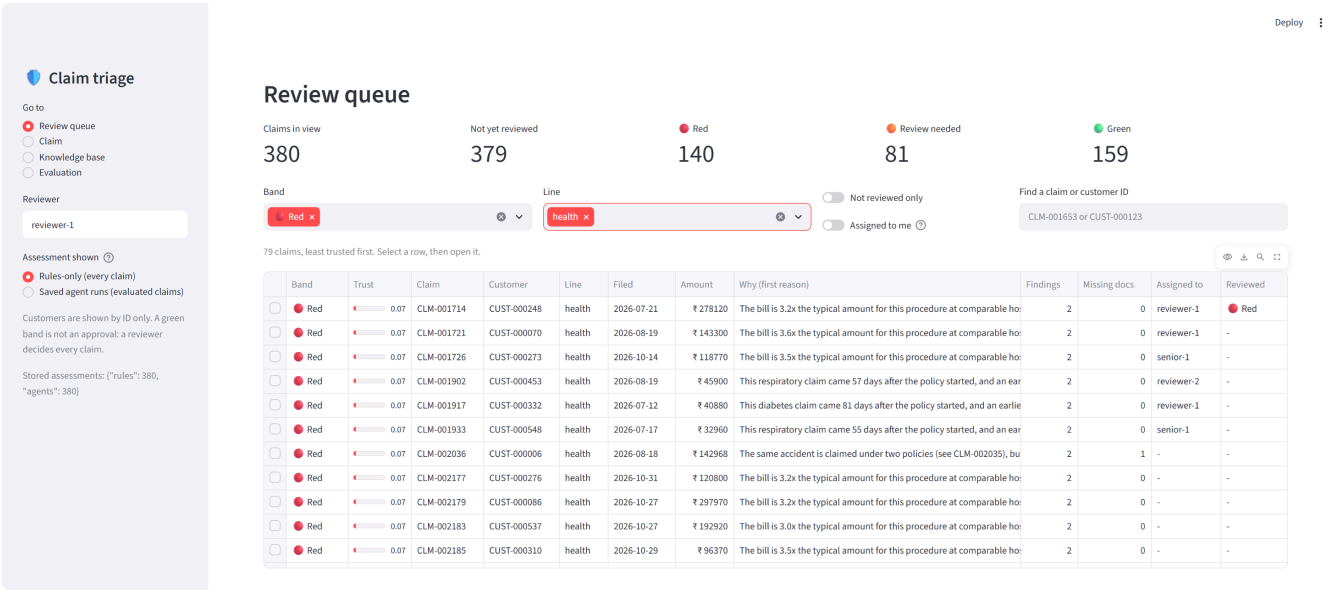
3. One claim, end to end

This walk-through follows claim CLM-001713 from the review queue to a saved decision and a proposed lesson, using the real app on a scratch database. The claim is a health claim for an appendectomy at Konda City Care Hospital, filed 2026-08-23 for 2,24,990 rupees by customer CUST-000496. The system rates it Red with a trust score of 0.10.

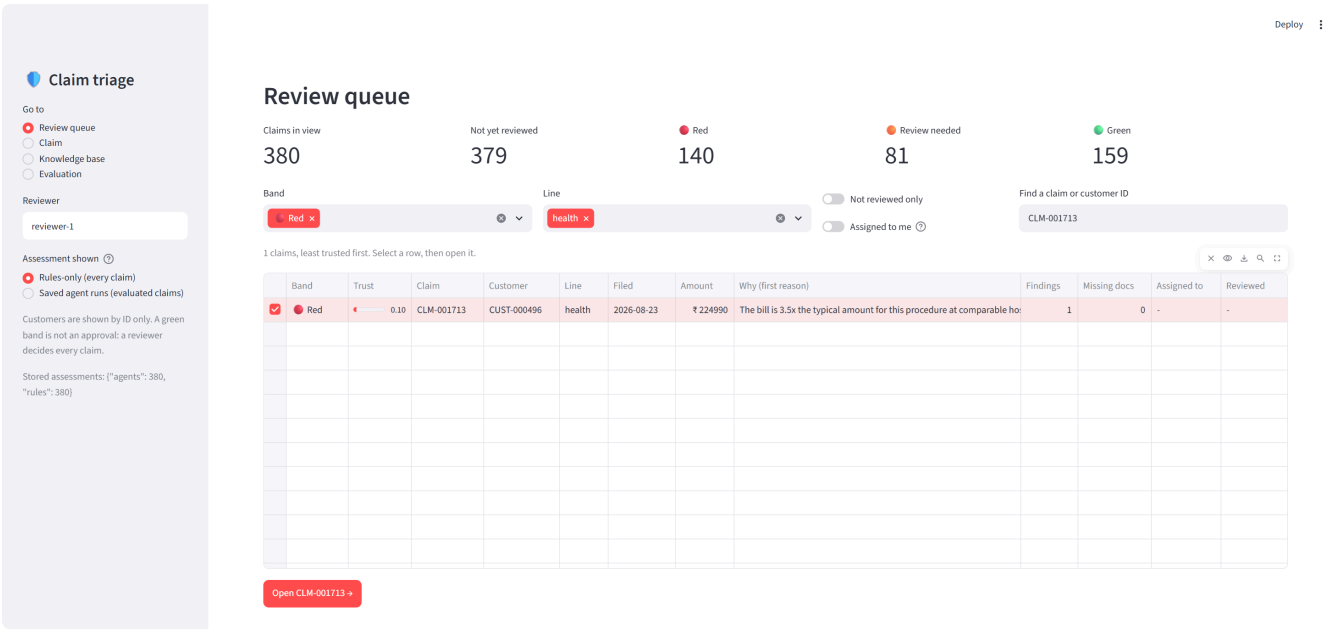
Step	Screen	What the reviewer does	What the system does
1	Review queue	filters to Red and health	narrows the list
2	Review queue	finds the claim, ticks its row, presses Open	stores the choice and switches to the Claim page

Step	Screen	What the reviewer does	What the system does
3	Claim	reads the score and reasons	shows 1 finding: inflated billing, confidence 100%
4	Claim	opens the cited records	shows the hospital record from the data
5	Claim	switches to the saved agent run	shows the Investigator's note and the second opinion
6	Claim	tries an invalid decision	refuses and explains
7	Claim	agrees, with a note	saves to the audit trail
8	Knowledge base	ticks the finding, adds a note, presses Update	saves it as pending and runs the safety check

Step 1. Narrow the queue. In the Band box choose Red and in the Line box choose health. The tiles stay at the totals; only the table is filtered.

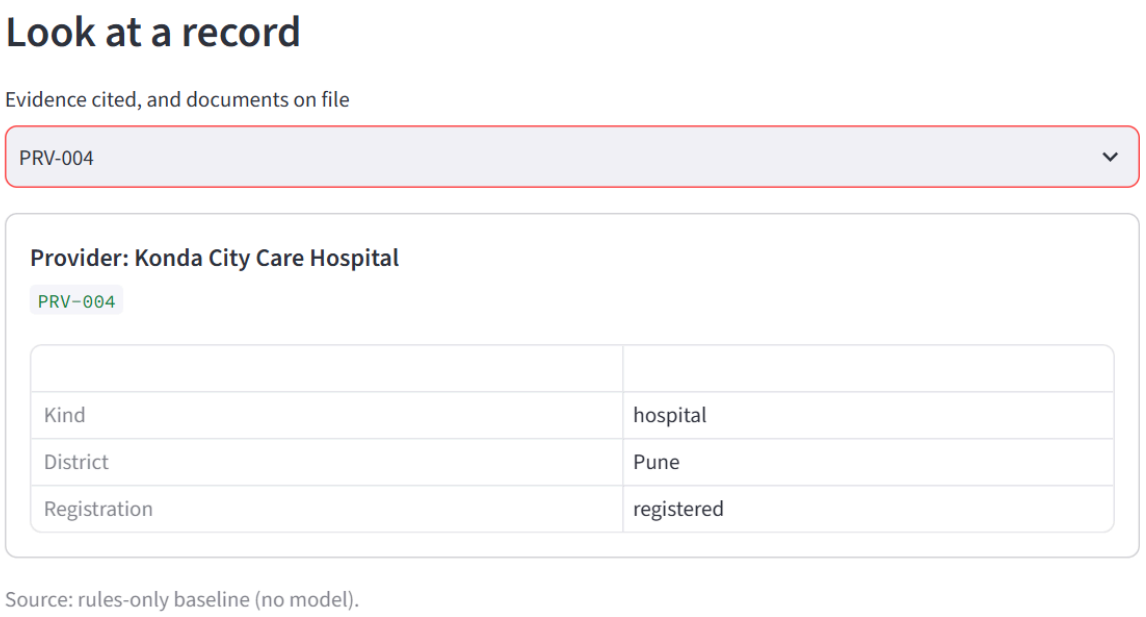


Step 2. Pick the claim and open it. Typing the claim ID in the search box leaves one row. Ticking its checkbox makes an "Open CLM-001713" button appear under the table.

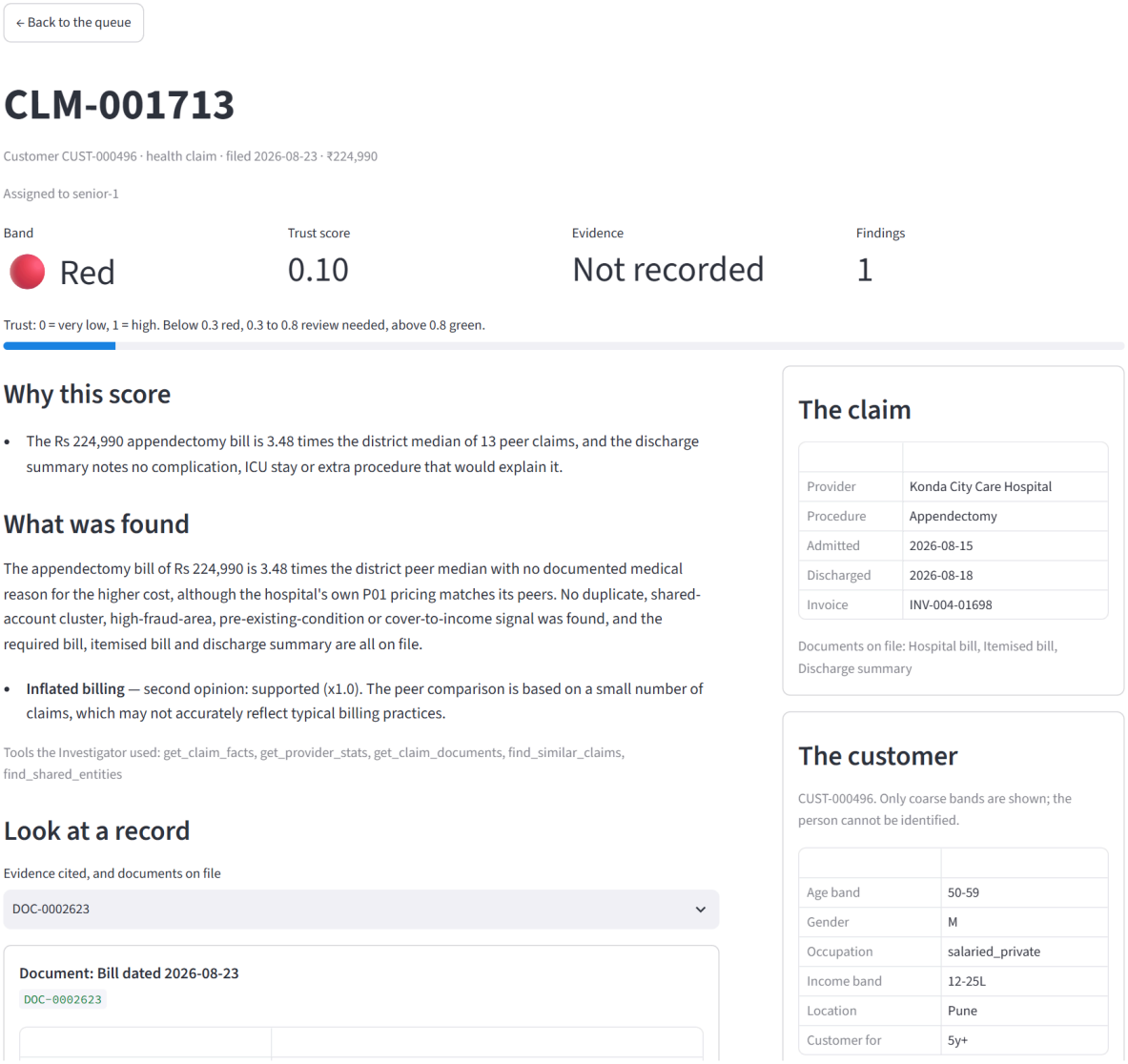


Step 3. Read the verdict. The Claim page opens on the score: Red, trust 0.10, evidence enough on file, 1 finding. "Why this score" gives the reason in one sentence: the bill is 3.5 times the typical amount for this procedure at comparable hospitals (13 comparable claims, district level). The full page is annotated in section 5, with every finding expandable and the suggested next step "Priority manual investigation".

Step 4. Check the evidence. "Look at a record" lists every ID the finding cites and every document on file. Choosing PRV-004 shows the provider: Konda City Care Hospital, a registered hospital in Pune. Choosing a document shows its scrubbed text. Nothing here can show a customer's name or phone number, because the page reads only the pseudonymous layer.



Step 5. Compare with the saved agent run. Switching "Assessment shown" in the sidebar to saved agent runs shows what the agents made of the same claim: the Investigator's summary (3.48 times the district median, no documented medical reason for the gap), the second opinion (supported, multiplier 1.0) and the five tools it used. The band and score are the same here.



Step 6. A decision that is refused. The reviewer chooses "Change the band" picks Review needed but writes no reason. Saving is refused with a message that says what to do, and nothing is stored.

Your decision

You decide this claim. The system's band is a recommendation, and disagreeing with it is fine; it is recorded for audit.

What do you decide?

- ○ Agree with the system's band
- ◉ Change the band
- ○ Hold: more information needed
- ○ Escalate to someone else

New band (used only if you change it)

Review needed

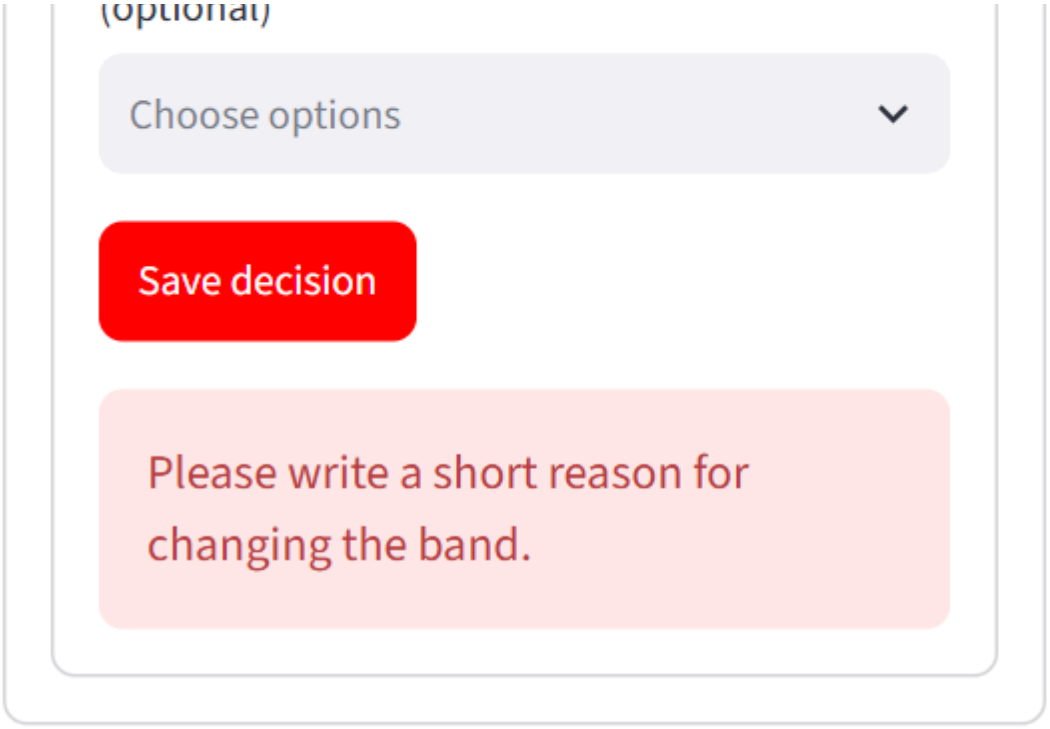
Escalate to (used only if you escalate)

Reviewer name, for example senior-1

Reason or note

Required if you change the band, hold the claim or escalate it.

What was wrong with the system's output? (optional)



Step 6b. Or hand it to someone else. Instead of deciding, reviewer-1 chooses "Escalate to someone else", types senior-1, writes why (the bill looks high but the rule is unclear) and ticks two tags: a policy or rule was misread, and the reasoning is unclear or wrong. No band is chosen. Saving shows "Escalated to senior-1. The claim stays open in the queue.", and the claim now reads "Assigned to senior-1". In the next step senior-1 takes over.

Your decision

You decide this claim. The system's band is a recommendation, and disagreeing with it is fine; it is recorded for audit.

What do you decide?

- ○ Agree with the system's band
- ○ Change the band
- ○ Hold: more information needed
- ◉ Escalate to someone else

New band (used only if you change it)

Review needed

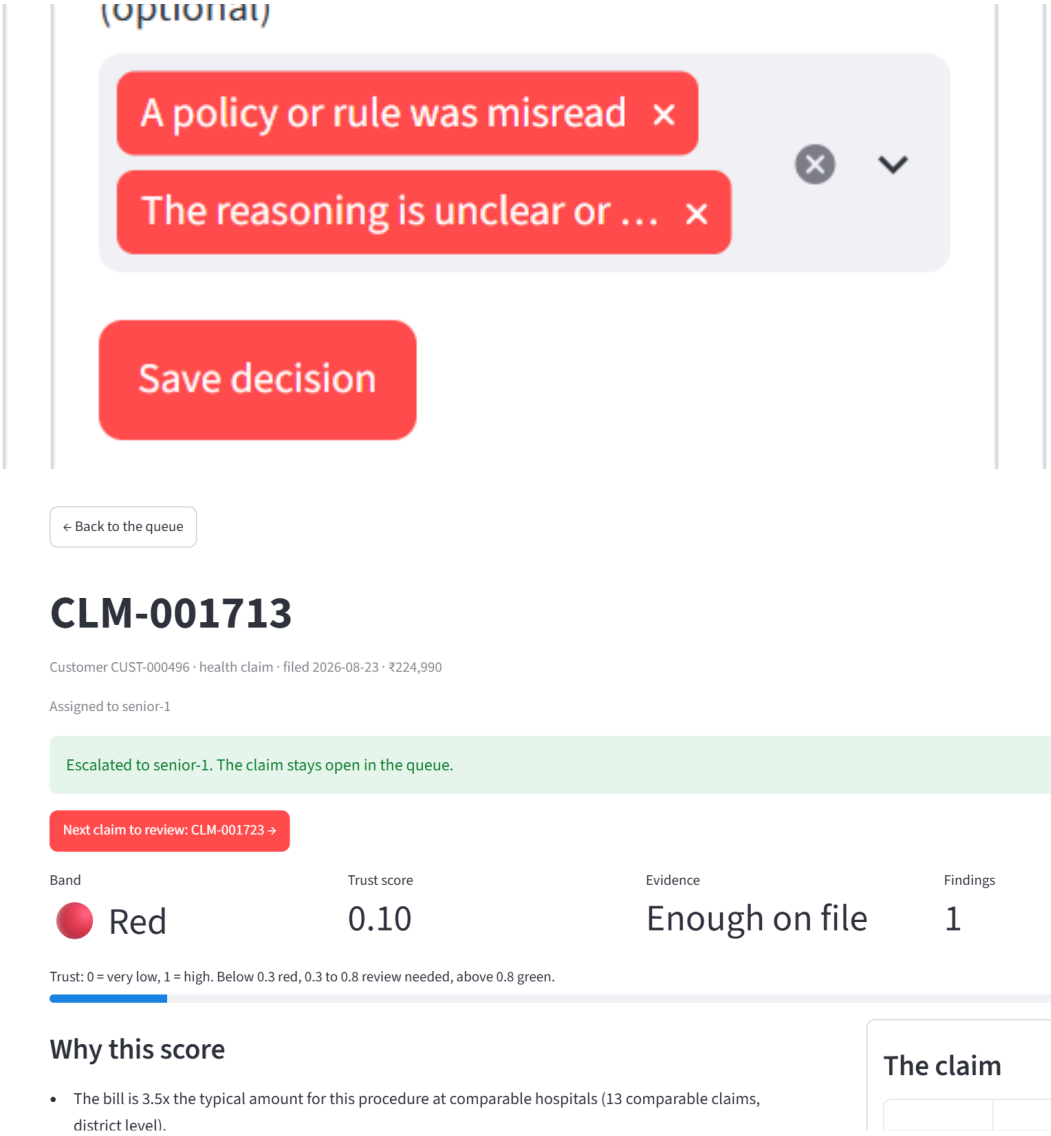
Escalate to (used only if you escalate)

senior-1

Reason or note

The bill looks high but I cannot tell if the procedure rule applies. Please take a look.

What was wrong with the system's output? (optional)



Step 7. Agree, with a note. The reviewer, now senior-1 who received the escalation, chooses "Agree with the system's band" and writes why: the bill is 3.5 times peers and the discharge summary gives no medical reason.

Your decision

You decide this claim. The system's band is a recommendation, and disagreeing with it is fine; it is recorded for audit.

- 05 Nov 10:08 · **reviewer-1** · Escalate to someone else to **senior-1**
  Tags: A policy or rule was misread, The reasoning is unclear or wrong
  The bill looks high but I cannot tell if the procedure rule applies. Please take a look.

What do you decide?

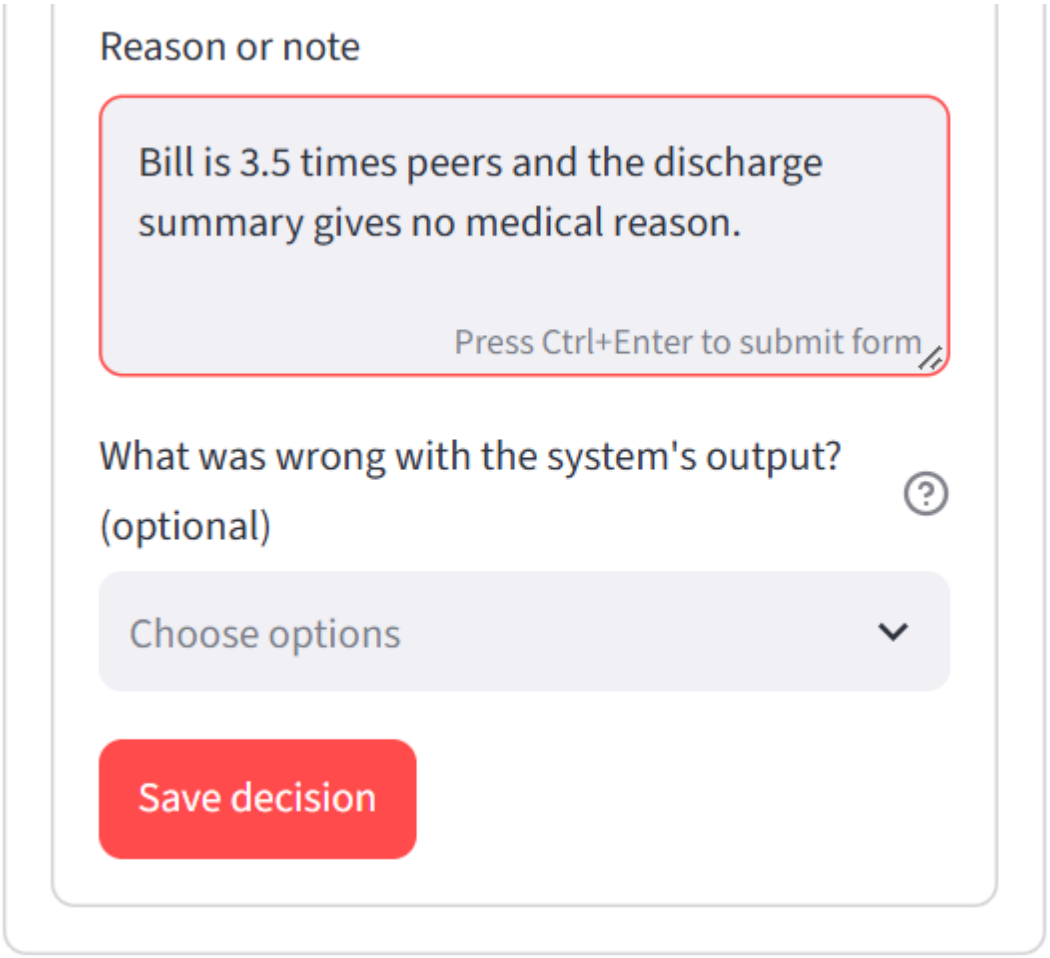
- (●) Agree with the system's band
- ( ) Change the band
- ( ) Hold: more information needed
- ( ) Escalate to someone else

New band (used only if you change it)

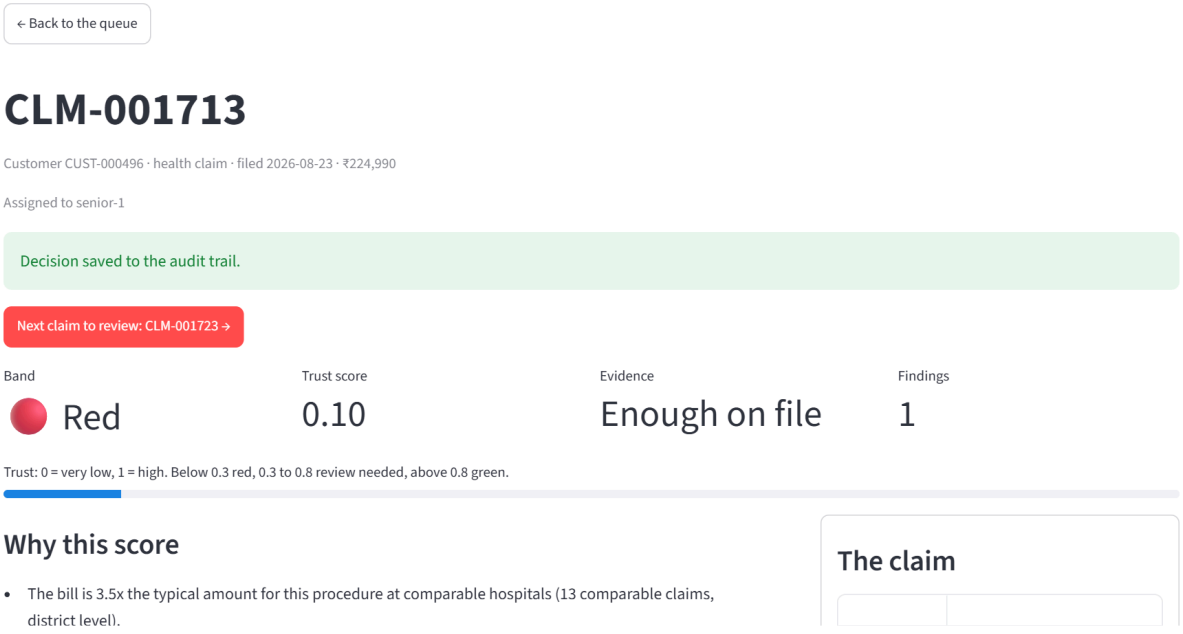
Red

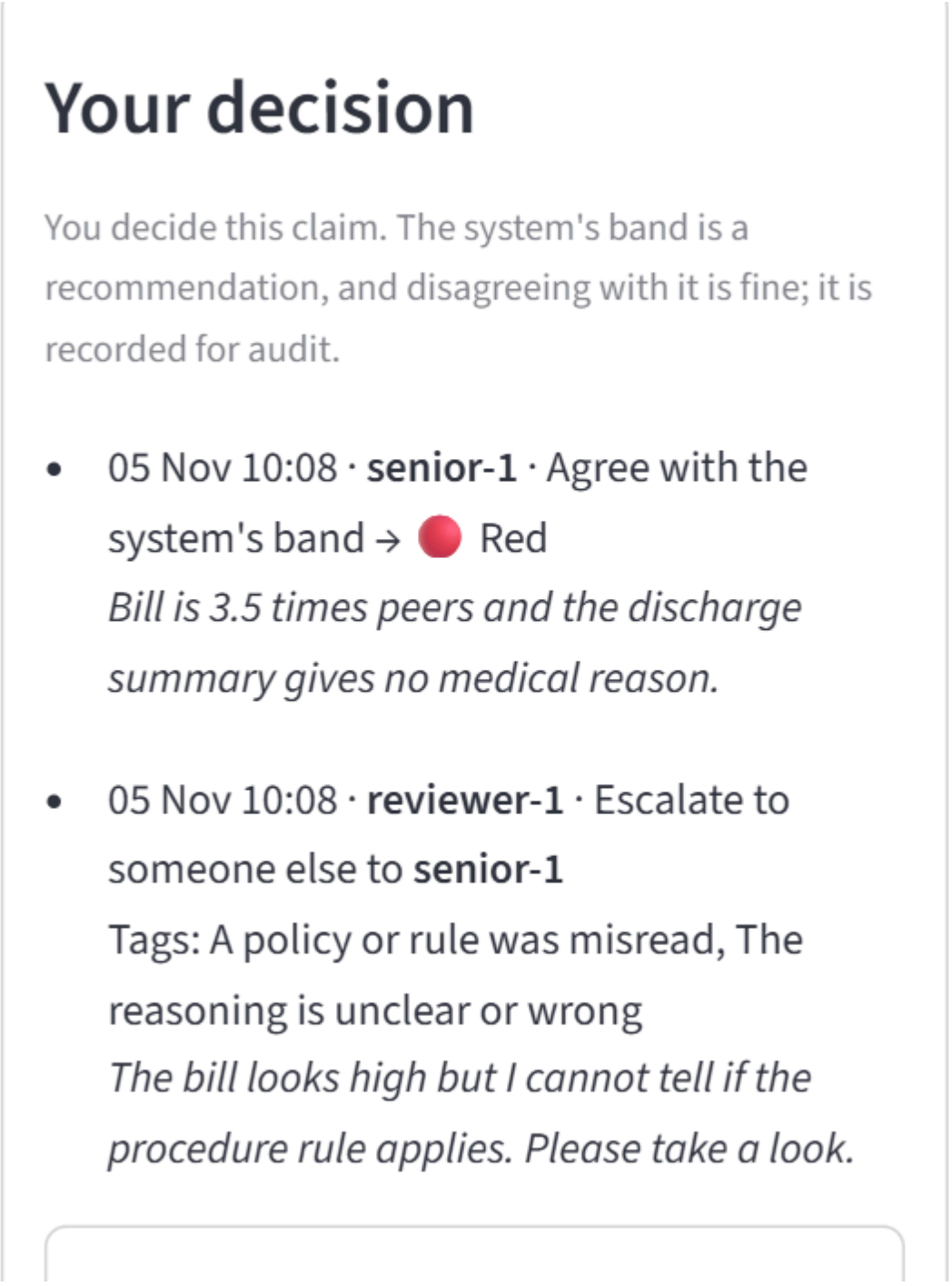
Escalate to (used only if you escalate)

Reviewer name, for example senior-1

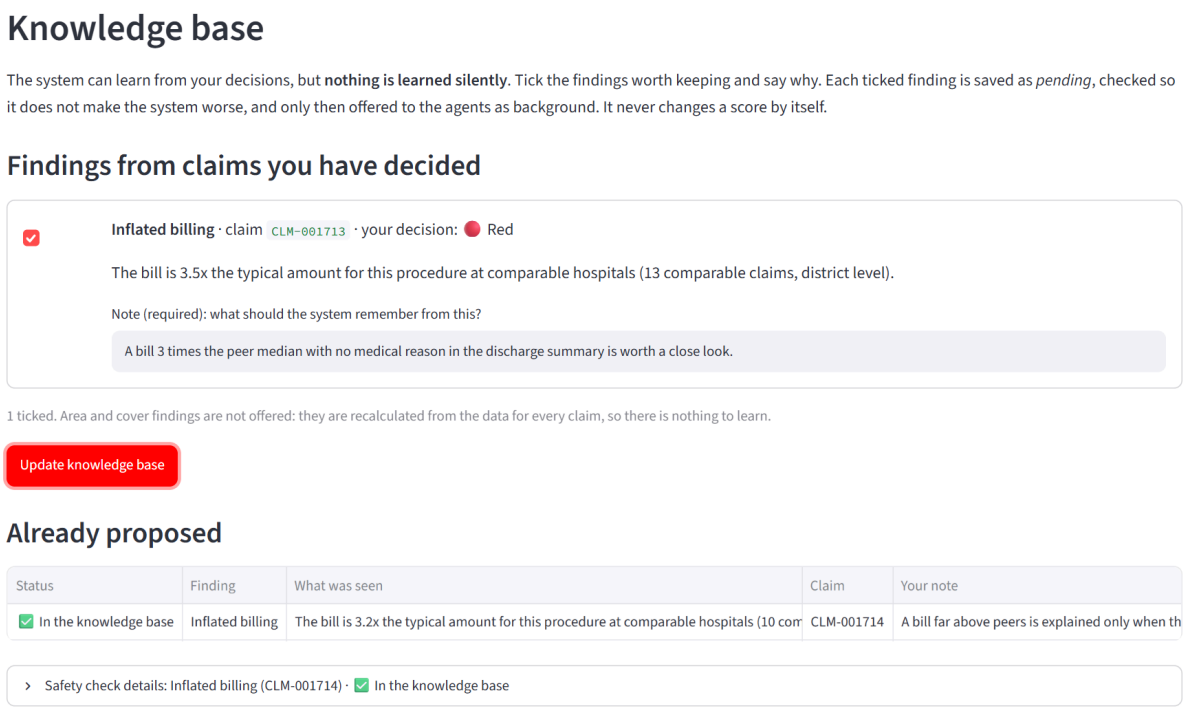


Saving shows a green confirmation and a button to the next unreviewed claim (CLM-001723). The decision appears in the panel's history with the time, the reviewer, the choice, the final band and the note.

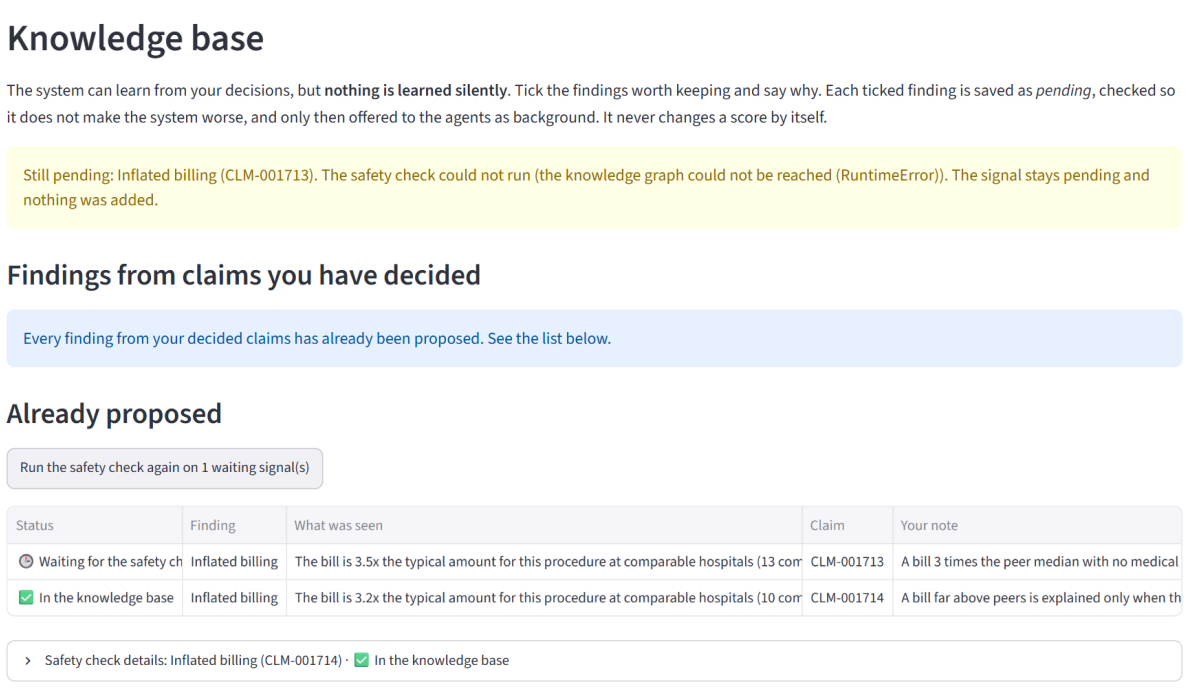




Step 8. Offer the finding as a lesson. On the Knowledge base page the finding from the decided claim is offered. The reviewer ticks it and writes what the system should remember (at least 15 characters).



Pressing "Update knowledge base" saves the lesson as pending and runs the safety check, which needs the Neo4j graph. **In this capture the free Aura instance was paused**, so the check could not run. The page says so in plain words, keeps the lesson as pending and adds nothing to the knowledge base. This is the designed fail-closed behaviour. Resuming the instance and pressing "Run the safety check again" would finish the job.



When the graph is reachable the same button ends with a green "Added to the knowledge base" banner. The earlier accepted lesson for CLM-001714 shows what the check looks like when it passes: nine checks, all ticked.

Status	Finding	What was seen	Claim	Your note
Waiting for the safety ch	Inflated billing	The bill is 3.5x the typical amount for this procedure at comparable hospitals (13 com	CLM-001713	A bill 3 times the peer median with no medical
In the knowledge base	Inflated billing	The bill is 3.2x the typical amount for this procedure at comparable hospitals (10 com	CLM-001714	A bill far above peers is explained only when th

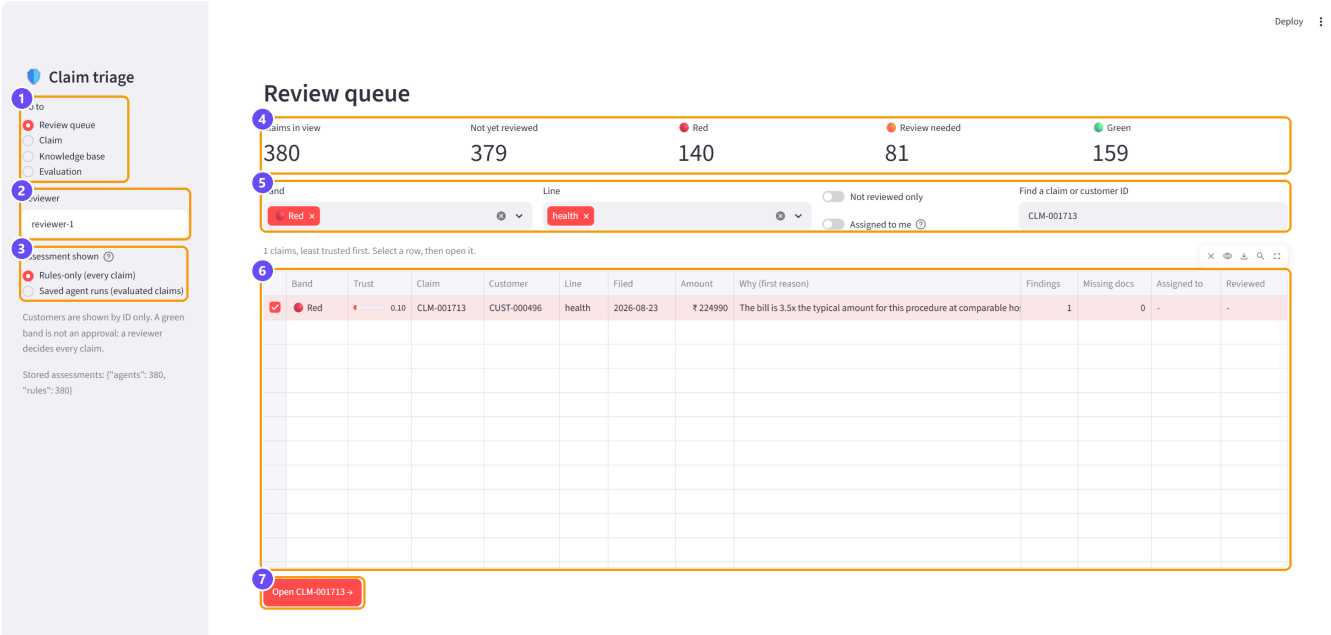
Safety check details: Inflated billing (CLM-001714) · In the knowledge base

- **cites real records**: 2 record(s) found
- **no personal data**: none found
- **nothing that reads like an instruction**: none found
- **no claims of certainty**: none found
- **a reasonable length**: 233 characters (limit 800)
- **not a copy of an existing signal**: new
- **keeps the right pattern card in the top 3**: no change for the top 3 in 36 test searches
- **does not show up in unrelated searches**: never

What changed in the data. One row in `decisions` (agree, Red, with the note, `disagrees_with_system` false) and one row in `signals` with status pending. The earlier escalation (step 6b) added its own row in decisions, with its tags. No score changed, no claim changed, and the graph was not touched.

4. Review queue

The queue shows every assessed claim, least trusted first, and the only way forward is to pick one row and open it. The page and the totals are computed from the assessment type chosen in the sidebar.

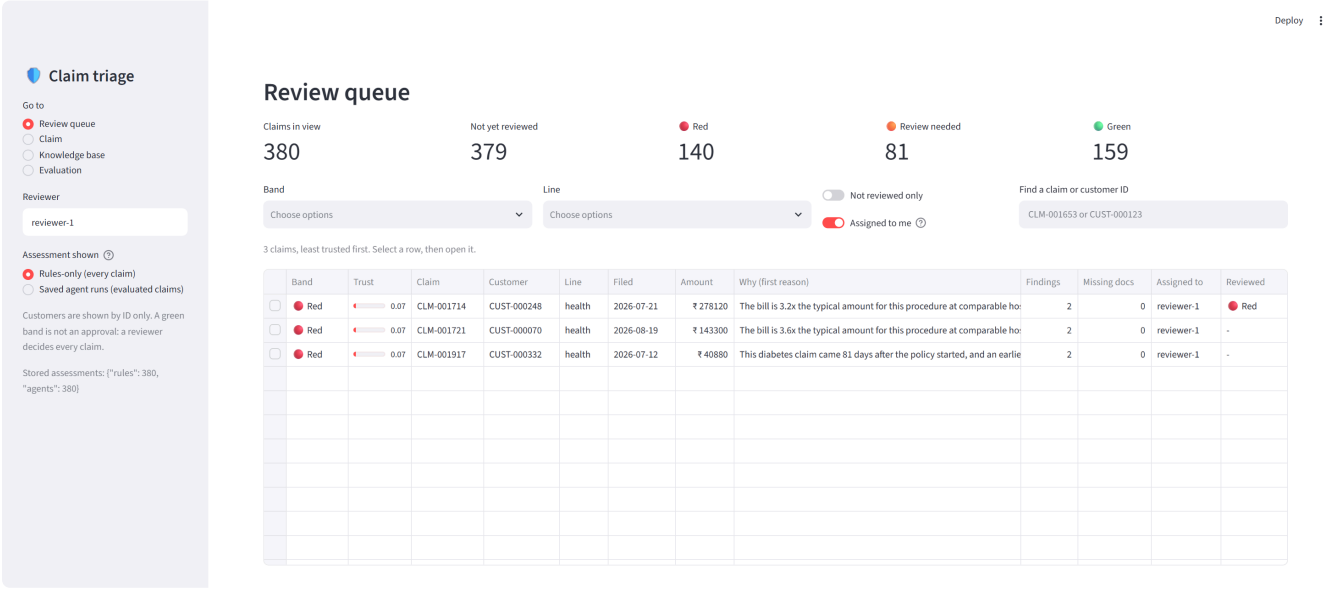


#	Component	What it shows or does	Source
1	**Go to** (sidebar)	Switches between Review queue, Claim, Knowledge base and Evaluation	`st.session_state["page"]`
2	**Reviewer** (sidebar)	The name saved on every decision and lesson. It must not be empty, or saving is refused. Default `reviewer-1`	`st.session_state["reviewer"]`
3	**Assessment shown** (sidebar)	"Rules-only (every claim)" or "Saved agent runs". It decides which stored assessment feeds the queue and the Claim page	`assessments.system`
4	**Summary tiles**	Claims in view, Not yet reviewed, Red, Review needed, Green. They count all claims of the chosen assessment, before the filters	`summarise()`
5	**Filters**	Band and Line multi-selects, a "Not reviewed only" toggle, an "Assigned to me" toggle and a search box that matches any part of a claim ID or customer ID	`apply_filters()`
6	**Claims table**	One row per claim: band, trust (a bar from 0 to 1), claim, customer ID, line, filed date, amount in rupees, the first reason, number of findings, missing documents, who the claim is assigned to and the band of the latest decision. Sorted by trust, then band, then claim ID. Select one row	`load_queue()`

#	Component	What it shows or does	Source
7	**Open button**	Appears once a row is ticked. Goes to the Claim page for that claim. If a claim was opened before and no row is ticked, a "Back to ..." button appears instead	`goto()`

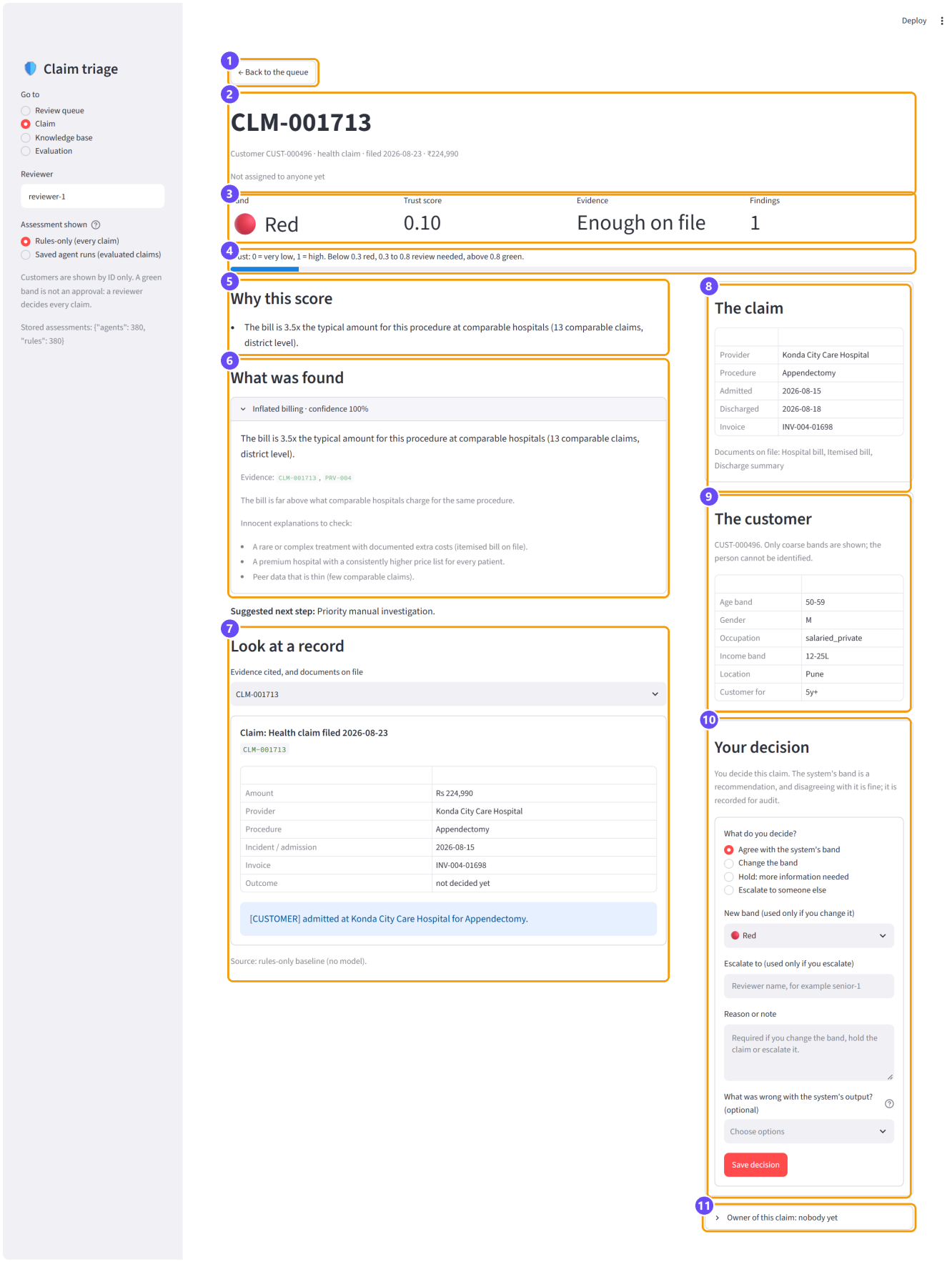
Behaviours worth knowing

- **Reviewed column.** It shows the final band of the claim's latest decision. A hold or an escalation does not count as a decision, so a held or escalated claim stays "not reviewed" and keeps showing in the open list.
- **Empty state.** With no stored assessments the page shows "No assessed claims yet" and the command to fill the database.
- **Customer privacy.** The table shows `CUST-` IDs only. Names, phones and exact locations are not in the data the page reads.
- **System label.** The sidebar option reads "Saved agent runs (evaluated claims)". Every one of the 380 evaluated claims has a saved run from the evaluation runs, as the stored counts in the sidebar show. The screenshots in this document were taken before the label changed from "some claims".



5. Claim page

The Claim page puts the verdict, the reasons, the proof and the decision form on one screen: a wide left column to read, a narrow right column to act in. It shows one claim at a time and opens from the queue or the "Next claim" button.


Deploy
Claim triage
Go to
Review queue
Claim
Knowledge base
Evaluation
Reviewer
reviewer-1
Assessment shown
Rules-only (every claim)
Saved agent runs (evaluated claims)
Customers are shown by ID only. A green band is not an approval: a reviewer decides every claim.
Stored assessments: {"agents": 380, "rules": 380}
1
← Back to the queue
2
CLM-001713
Customer CUST-000496 · health claim · filed 2026-08-23 · ₹224,990
Not assigned to anyone yet
3
Band
Red
Trust score
0.10
Evidence
Enough on file
Findings
1
4
Trust: 0 = very low, 1 = high. Below 0.3 red, 0.3 to 0.8 review needed, above 0.8 green.
5
Why this score
The bill is 3.5x the typical amount for this procedure at comparable hospitals (13 comparable claims, district level).
6
What was found
Inflated billing · confidence 100%
The bill is 3.5x the typical amount for this procedure at comparable hospitals (13 comparable claims, district level).
Evidence: CLM-001713, PRV-004
The bill is far above what comparable hospitals charge for the same procedure.
Innocent explanations to check:
A rare or complex treatment with documented extra costs (itemised bill on file).
A premium hospital with a consistently higher price list for every patient.
Peer data that is thin (few comparable claims).
Suggested next step: Priority manual investigation.
7
Look at a record
Evidence cited, and documents on file
CLM-001713
Claim: Health claim filed 2026-08-23
CLM-001713
Amount
Rs 224,990
Provider
Konda City Care Hospital
Procedure
Appendectomy
Incident / admission
2026-08-15
Invoice
INV-004-01698
Outcome
not decided yet
[CUSTOMER] admitted at Konda City Care Hospital for Appendectomy.
Source: rules-only baseline (no model).
8
The claim
Provider
Konda City Care Hospital
Procedure
Appendectomy
Admitted
2026-08-15
Discharged
2026-08-18
Invoice
INV-004-01698
Documents on file: Hospital bill, Itemised bill, Discharge summary
9
The customer
CUST-000496. Only coarse bands are shown; the person cannot be identified.
Age band
50-59
Gender
M
Occupation
salaried_private
Income band
12-25L
Location
Pune
Customer for
5y+
10
Your decision
You decide this claim. The system's band is a recommendation, and disagreeing with it is fine; it is recorded for audit.
What do you decide?
Agree with the system's band
Change the band
Hold: more information needed
Escalate to someone else
New band (used only if you change it)
Red
Escalate to (used only if you escalate)
Reviewer name, for example senior-1
Reason or note
Required if you change the band, hold the claim or escalate it.
What was wrong with the system's output? (optional)
Choose options
Save decision
11
Owner of this claim: nobody yet

#	Component	What it shows or does	Source
1	**Back to the queue**	Returns to the Review queue	`goto()`
2	**Header**	Claim ID, then customer ID, line, filing date and amount, then who the claim is assigned to (or "Not assigned to anyone yet"). After a save, a green confirmation and a "Next claim to review" button appear here	`claim_overview()`, `next_unreviewed()`
3	**Four tiles**	Band; Trust score; Evidence ("Enough on file", "Missing or not checked" or "Not recorded"); Findings (a count)	the stored assessment
4	**Trust bar**	A bar from 0 to 1 with the band thresholds in words. For green, an extra note says green is not an approval	`st.progress`
5	**Why this score**	The reasons in plain sentences, in order of importance (the Explainer's wording for agents, code sentences for rules)	`result["reasons"]`

#	Component	What it shows or does	Source
6	**What was found**	One expandable block per finding, named in plain words with its confidence, open by default from 50%. Inside: the explanation, any hard-rule note, "could be innocent" counter-evidence, the evidence IDs and the pattern card's description. Below: "Still to check" (missing documents), the suggested next step and, if relevant, what would turn the claim green	`findings_section()`, `plain.finding_note()`
7	**Look at a record**	A drop-down of every cited ID and every document on file. Choosing one shows its card: kind, title, ID, a fact table and the scrubbed text. A "Source" line says where the assessment came from	`EvidenceLookup.describe()`
8	**The claim**	Provider, procedure, admission and discharge dates, incident date and invoice number (only those that exist), plus the documents on file	`claim_overview()`
9	**The customer**	Coarse bands only: age band, gender, occupation, income band, location (district, state or "withheld") and tenure	`customer_facts()`

#	Component	What it shows or does	Source
10	**Your decision**	The history of decisions on this claim, then a form: a four-way choice (agree, change the band, hold, escalate), a band drop-down, an "Escalate to" name, a note, optional tags and Save	`decision_panel()`, `record_decision()`
11	**Owner of this claim**	A closed box under the decision panel. Opened, it lets a reviewer give the claim to someone by name, or clear the owner. An escalation fills it in automatically	`owner_panel()`, `Store.assign()`

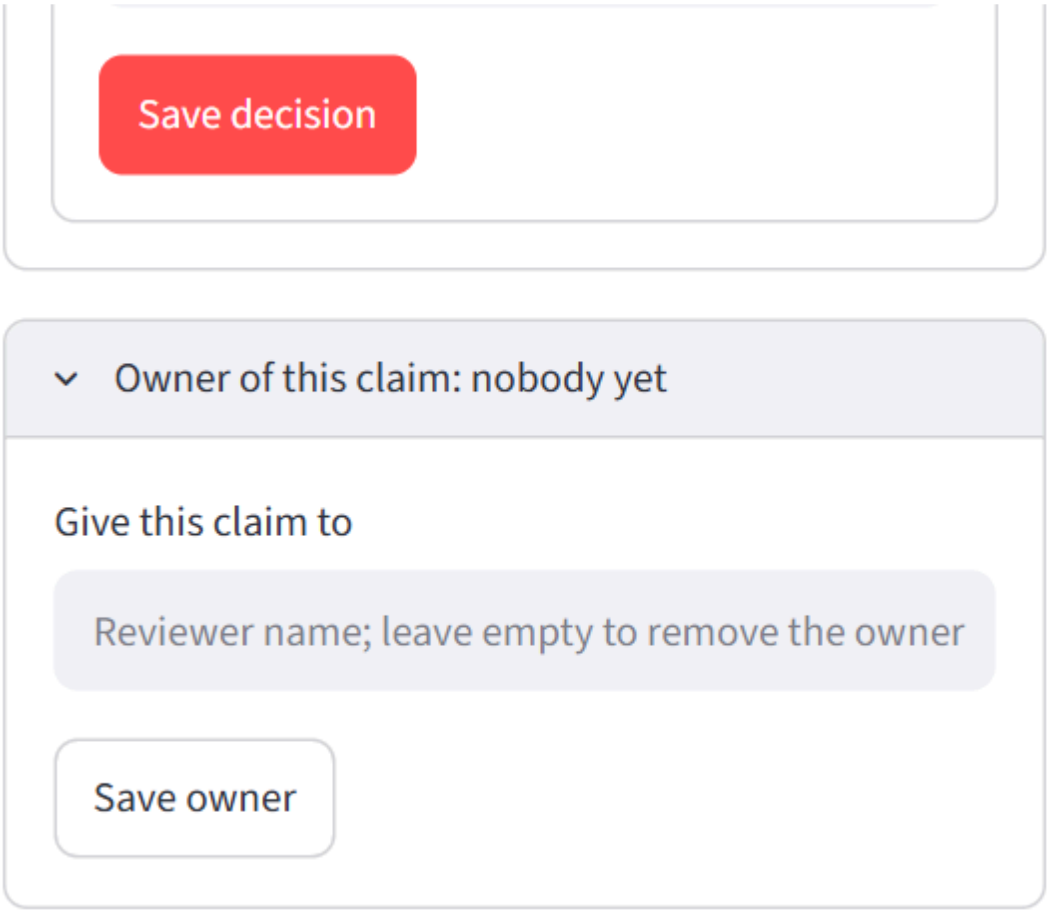
What the page looks like for a saved agent run. The "What was found" block lists what the Investigator proposed, each with the Verifier's verdict, multiplier and reason, and the tools the Investigator used. The Evidence tile reads "Not recorded" because saved runs do not store that field. If a claim has no saved run, the page falls back to the rules-only result and says so.

The four decisions

Choice	Meaning	Rules
Agree with the system's band	The reviewer accepts the band	The final band is the system's band; a note is optional
Change the band	The reviewer sets a different band	The new band must differ from the system's and a note is required
Hold: more information needed	The claim stays open	A note saying what is needed is required; the claim still counts as not reviewed
Escalate to someone else	The reviewer hands the claim to another reviewer; the claim stays open and now belongs to that person	The other reviewer's name (not your own) and a note saying why are required; the system's band is kept and the claim still counts as not reviewed

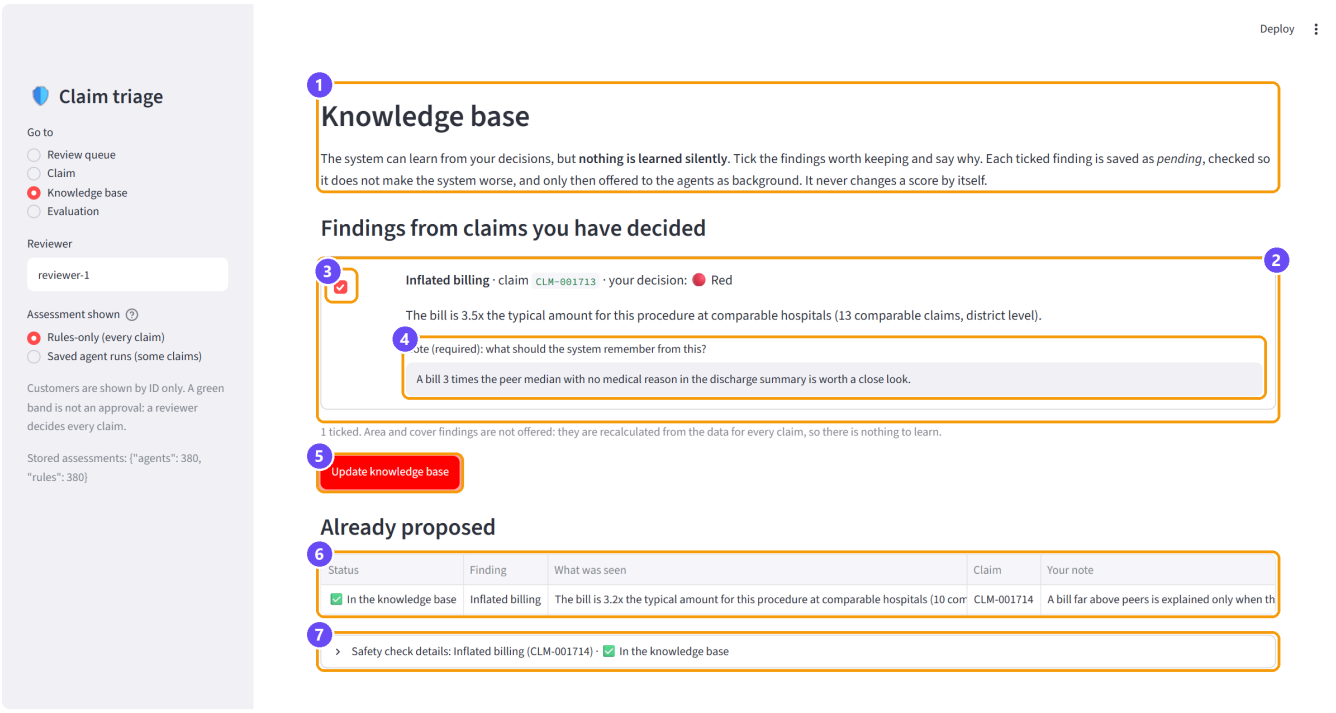
The reviewer name in the sidebar must not be empty for any choice. Every saved decision records whether it disagreed with the system, and disagreeing is never blocked.

Tags and owner. Under the note box a reviewer can tick optional tags saying what was wrong with the system's output: the evidence did not support the finding, evidence I needed was not shown, the reasoning is unclear or wrong, a policy or rule was misread, or the wording is off. Tags are saved with the decision and can later become evaluation labels. Below the decision panel an *Owner of this claim* box lets a reviewer give the claim to someone by name (or clear the owner). The owner is shown under the claim title, in an *Assigned to* column of the queue, and an *Assigned to me* switch keeps only the claims owned by the reviewer name in the sidebar. An escalation sets the owner automatically.



6. Knowledge base page

This page is where a reviewer teaches the system, and it is built so that nothing is learned silently: every lesson needs a decision, a written note and a passed safety check. A lesson only becomes background for the agents; it never changes a score.



#	Component	What it shows or does	Source
1	**Introduction**	States the rule: ticked findings are saved as pending, checked, and only then offered to the agents as background	static text
2	**Finding card**	One card per finding of every claim the reviewer has decided (holds and escalations do not count), newest decision first, at most 50 shown. It names the finding, the claim, the reviewer's final band and the explanation	`candidate_signals()`
3	**Tick box**	"Add to the knowledge base". Area and cover findings never appear, because they are recalculated from the data for every claim	`NOT_LEARNABLE`

#	Component	What it shows or does	Source
4	**Note**	Required and disabled until the box is ticked: at least 15 characters, or 50 if the reviewer contradicted the system on that claim (an orange line says so). A finding blocked earlier is offered again with a note that it may be retried	`submit_signals()`
5	**Update knowledge base**	Disabled until something is ticked. Saves every ticked finding as pending, then runs the safety check under a spinner	`submit_signals()`, `run_safety_check()`
6	**Already proposed**	A table of every lesson: status (waiting for the check, in the knowledge base, or blocked), finding, what was seen, claim, the note, who proposed it, when, an audit tag for contradictions and the check's summary. A button reruns the check on waiting lessons	`signal_rows()`
7	**Safety check details**	One expandable block per lesson with every check marked passed or failed, with its detail. A lesson that passes shows nine: six on its text and three on the search	`GateResult`

Result banners. After a run, a banner per lesson appears at the top: green "Added to the knowledge base", red "Not added" with the reason, or amber "Still pending" when the check could not run (for example, the graph was unreachable).

Statuses

Status shown	Stored as	Meaning
Waiting for the safety check	`pending`	Saved, not yet checked, or the check could not run. Invisible to agents
In the knowledge base	`active`	Passed. Agents can find it when they search
Blocked by the safety check	`blocked`	Failed. Can be proposed again, with a better note

A lesson that is pending or active cannot be proposed twice.

7. Evaluation page

The Evaluation page is the team lead's page: it lists every test run with what it does and does not prove, and it opens with a warning that the plain rules beat the agents on clean data. It reads only the result CSV files, never the answer keys.

Deploy

Claim triage

Go to
- Review queue
- Claim
- Knowledge base
- Evaluation

Reviewer: reviewer-1

Assessment shown
- Rules-only (every claim)
- Saved agent runs (some claims)

Customers are shown by ID only. A green band is not an approval: a reviewer decides every claim.

Stored assessments: {"agents": 380, "rules": 380}

Evaluation

1

Read this first. Everything below comes from tests on made-up data. The AI agents were run on every dev and held-out case with the small model, and the two agree. **On the original clean data the plain rules beat the agents (99% to 99.5% against 91.5% to 93%).** The agents win only on the hard cases, which were designed to need reading text and looking up a hospital's record. The larger model closes most of the gap on the two weakest scenarios, at about 18 times the cost per claim. Each line says what it does and does not show.

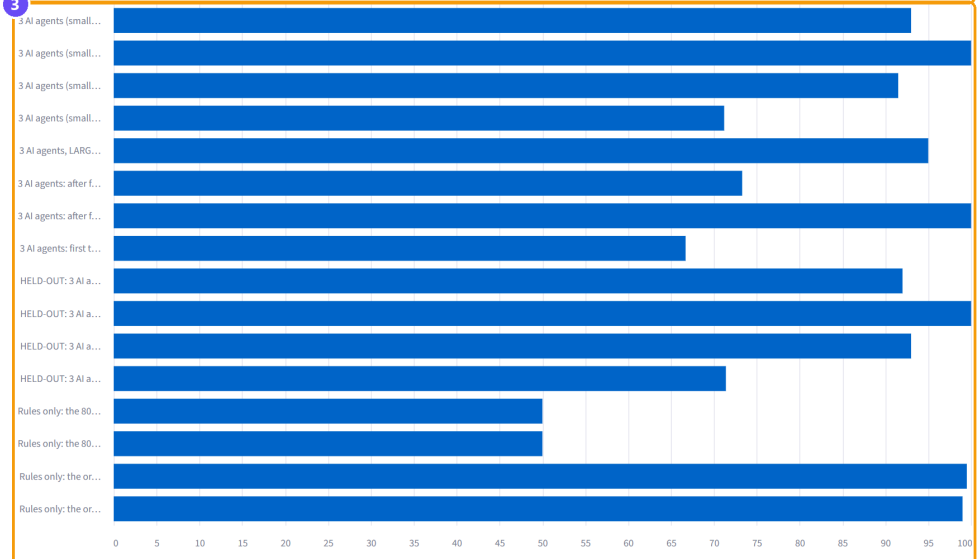
Runs so far

2

Run	System	Cases	Right band (%)	Fraud called green (%)	Honest called red (%)	Cost ($)	What
Rules only: the original 300 cases (dev)	Rules (no AI)	200	99.5	0	0	None	No A
Rules only: the original 300 cases (held-out)	Rules (no AI)	100	99.0	0	0	None	No A
Rules only: the 80 hard cases (dev)	Rules (no AI)	52	50.0	50	50	None	Rule
Rules only: the 80 hard cases (held-out)	Rules (no AI)	28	50.0	50	50	None	Rule
3 AI agents: first try (30 original dev cases)	3 AI agents	30	66.7	0	0	1.22	The
3 AI agents: after fix 1 (same 30 cases)	3 AI agents	30	73.3	0	0	1.22	The
3 AI agents: after fix 2 (same 30 cases)	3 AI agents	30	100.0	0	0	1.22	Look
3 AI agents (small model), no retrieval: all 200 original dev cases	3 AI agents, no retrieval	200	91.5	0	0	0.64	First
3 AI agents (small model) with retrieval: all 200 original dev cases	3 AI agents + retrieval	200	93.0	0	0	0.77	Sam
3 AI agents (small model), no retrieval: all 52 hard dev cases	3 AI agents, no retrieval	52	71.2	50	0	0.17	Read

Right band: the system's band (red, review needed, green) matched what the test expected. Fraud called green is the number that matters most: it should be near 0. Honest called red is the cost of over-flagging. Agent runs were made on earlier builds of the data (the original 300 claims are unchanged).

3



4

The hard cases: what each setup can and cannot do

Setup	big bill, reason in the text (dev)	slightly high bill, bad hospital record (dev)	big bill, reason in the text (held-out)	slightly high bill, bad hospital record (held-out)
Rules only	13 of 26	13 of 26	7 of 14	7 of 14
3 AI agents, no retrieval	24 of 26	13 of 26	13 of 14	7 of 14
3 AI agents with retrieval	26 of 26	26 of 26	14 of 14	14 of 14

The first kind can only be solved by reading the discharge summary; the second only by looking up the hospital's record of confirmed frauds. These cases were designed to need exactly those abilities, so they show what the abilities are worth, not how often they matter in real claims.

5

Look closer at one run

Run: Rules only: the original 300 cases (dev)

What the test expected (rows) vs what the system said (columns)

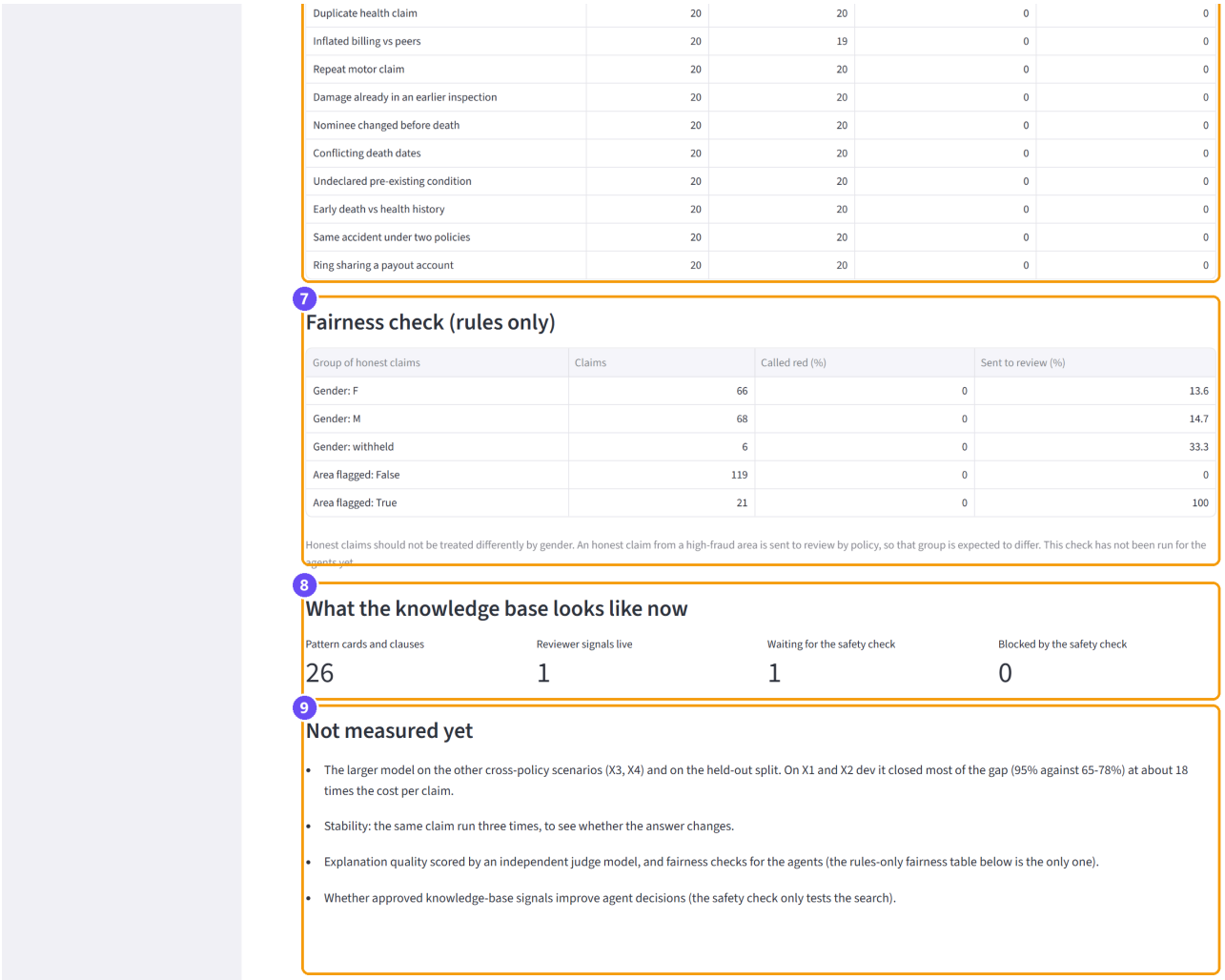
	red	amber	green
expected red	69	1	0
expected amber	0	51	0
expected green	0	0	79

Totals

Cases	200
Right band	99.5%
Fraud flagged (red or review)	100.0%
Claims the system could not assess	0
Reasons that fell back to the original sentences	•
Cost per case	none (no AI)
Median time per claim	instant

6

Scenario	Cases	Right band	Fraud called green	Honest called red



Duplicate health claim	20	20	0	0
Inflated billing vs peers	20	19	0	0
Repeat motor claim	20	20	0	0
Damage already in an earlier inspection	20	20	0	0
Nominee changed before death	20	20	0	0
Conflicting death dates	20	20	0	0
Undeclared pre-existing condition	20	20	0	0
Early death vs health history	20	20	0	0
Same accident under two policies	20	20	0	0
Ring sharing a payout account	20	20	0	0

#	Component	What it shows or does	Source
1	**Read this first**	A warning that all numbers come from tests on made-up data, that the rules beat the agents on clean data (99 to 99.5% against 91.5 to 93%), and where the agents win	static text
2	**Runs so far**	One row per run: name, system, cases, right band (with a bar), fraud called green, honest called red, cost, and a plain sentence on what the run tells us. A caption defines each measure	`load_runs()`
3	**Bar chart**	The right-band percentage of every run, side by side	the table above

#	Component	What it shows or does	Source
4	**The hard cases**	For rules only, agents without retrieval and agents with retrieval, how many of the two hard probes were right on dev and held-out	`per_scenario()`
5	**Look closer at one run**	A drop-down of runs. For the chosen run: a grid of expected band (rows) against the system's band (columns), and totals (cases, right band, fraud flagged, claims that could not be assessed, how many reasons fell back to the original sentences, cost and median time per claim)	`confusion()`, `metrics()`
6	**Per-scenario table**	For the chosen run, each fraud scenario with its cases, right band, fraud called green and honest called red	`per_scenario()`
7	**Fairness check (rules only)**	How often honest claims land red or in review, by gender and by whether the area was flagged. It says the check has not been run for the agents	`fairness()`
8	**Knowledge base counters**	Pattern cards and clauses, signals live, waiting for the check, blocked	`knowledge_counts()`, `signals`
9	**Not measured yet**	The honest list of gaps: for example the larger model on the other scenarios, run-to-run stability and explanation quality	`NOT_MEASURED`

What each measure means

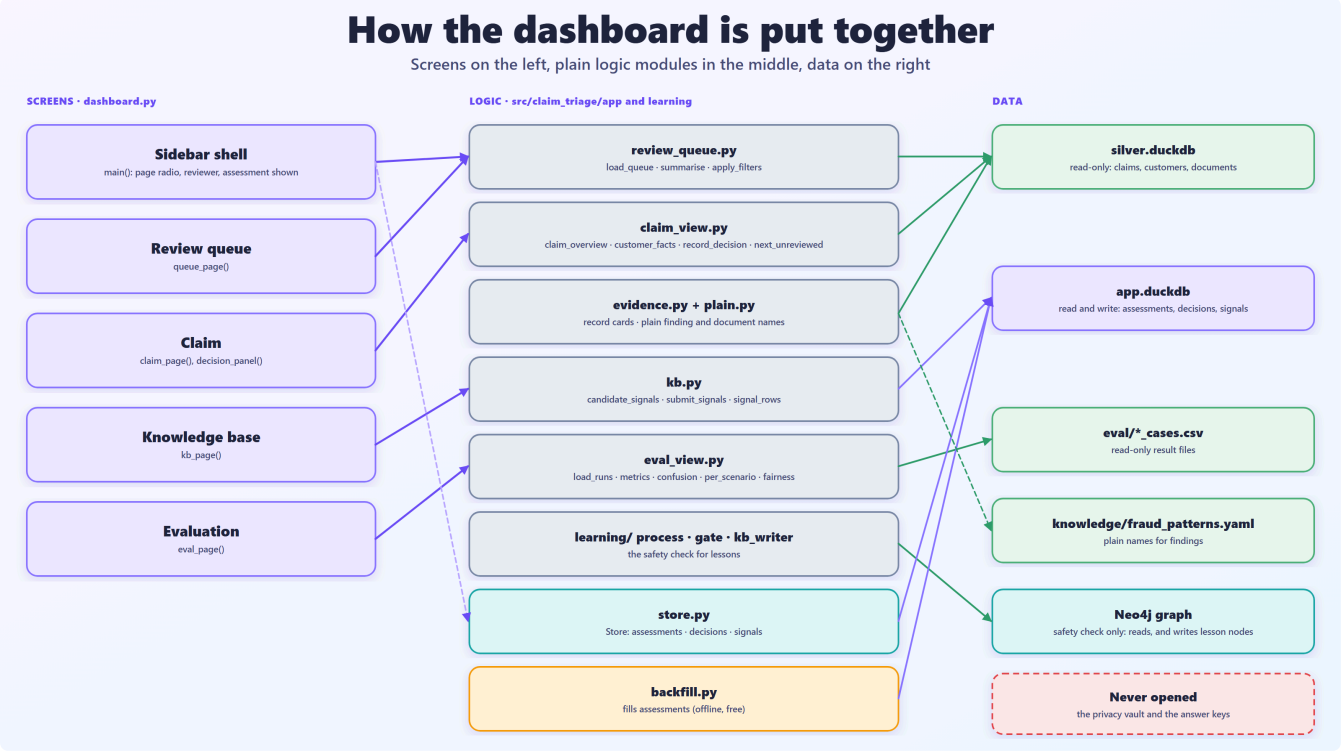
Measure	Plain meaning
Right band	The system's colour matched what the test expected for that claim

Measure	Plain meaning
Fraud called green	The number that matters most; it should be near zero
Honest called red	The cost of over-flagging
Reasons that fell back	How often the Explainer's rewrite was not usable and the original sentences were kept. Scores are never affected

Design rules. The page shows totals, never the expected answer for one claim. A test checks that the page's source never references the answer-key database. Trivial trial runs are hidden, and each remaining run carries a note that states its limits.

8. Logical components

The dashboard is one Streamlit script, `dashboard.py`, that only draws screens; every rule lives in small plain-Python modules that can be tested without a browser. The screens call the modules, and the modules read and write a few files.



Module	Responsibility	Reads	Writes
`app/dashboard.py`	Page shell and the four pages: widgets, layout, navigation, session state	everything below, through the modules	nothing directly
`app/review_queue.py`	Builds the ranked queue, the summary counts and the filters	`assessments`, `decisions`, `silver.duckdb`	nothing
`app/claim_view.py`	The claim and customer views, validation and saving of a decision, and the next claim to review	`silver.duckdb`, `decisions`	`decisions` (through the store)
`app/evidence.py`	Turns an evidence ID into a readable record card (claim, document, event, provider, policy or customer)	`silver.duckdb`	nothing
`app/plain.py`	Plain names for findings and documents, and the pattern card text	`knowledge/fraud_patterns.yaml`	nothing
`app/kb.py`	Which findings may be offered as lessons, validation of a proposal, and the signal list	`decisions`, `assessments`, `signals`	`signals` (through the store)
`app/eval_view.py`	Reads the result CSVs and computes the metrics, grids and fairness table shown on the Evaluation page	`eval/*_cases.csv`, `knowledge/`	nothing

Module	Responsibility	Reads	Writes
`app/store.py`	The app database: four tables and their access methods	`app.duckdb`	`app.duckdb`
`app/backfill.py`	Offline command that fills `assessments` (rules for every claim, saved agent runs from `traces/`)	gold, silver, `traces/`	`assessments`
`learning/process.py`, `gate.py`, `kb_writer.py`	The safety check for lessons and the only code that writes to the graph	`signals`, the Neo4j graph	`signals`, `ApprovedSignal` nodes

Design choices

- **Logic outside the screens.** Because each rule is a plain function, the app tests call them directly and also run the whole page headlessly.
- **A separate app database.** Rebuilding the data never erases a reviewer's decision, and the page never needs the vault or the answer keys.
- **Open and close per run.** Streamlit reruns the script on every interaction, so each run opens the store, draws the page and closes the store in a `finally` block.
- **Two assessment types behind one page.** The queue and the Claim page read `assessments` by system, so a saved agent run uses the same screens as the rules-only result.

9. Application logic

State lives in Streamlit's session, decisions live in the app database, and navigation is a button callback that sets the page before the script redraws. Everything below is what the code does today.

Session state

Key	Holds	Set by
`page`	the current page; also the sidebar radio's own key	the radio, or the `goto()` callback

Key	Holds	Set by
`claim_id`	the claim open on the Claim page	a table selection, `goto()` or the Next claim button
`reviewer`	the reviewer name from the sidebar text box	the text input
`saves`	a counter that makes the decision form's key new after each save, so the form resets	`decision_panel()`
`just_saved`	the claim whose decision was just saved, to show the confirmation once	`decision_panel()`
`kb_results`	the result banners of the last safety-check run, shown once	the Knowledge base page
`kb_n`	a counter that resets the tick boxes and notes after an update	the Knowledge base page
widget keys	`queue-<system>` for the table, `record-<claim>` for the record drop-down, `kb-add-<n>-<key>` and `kb-note-<n>-<key>`	the widgets

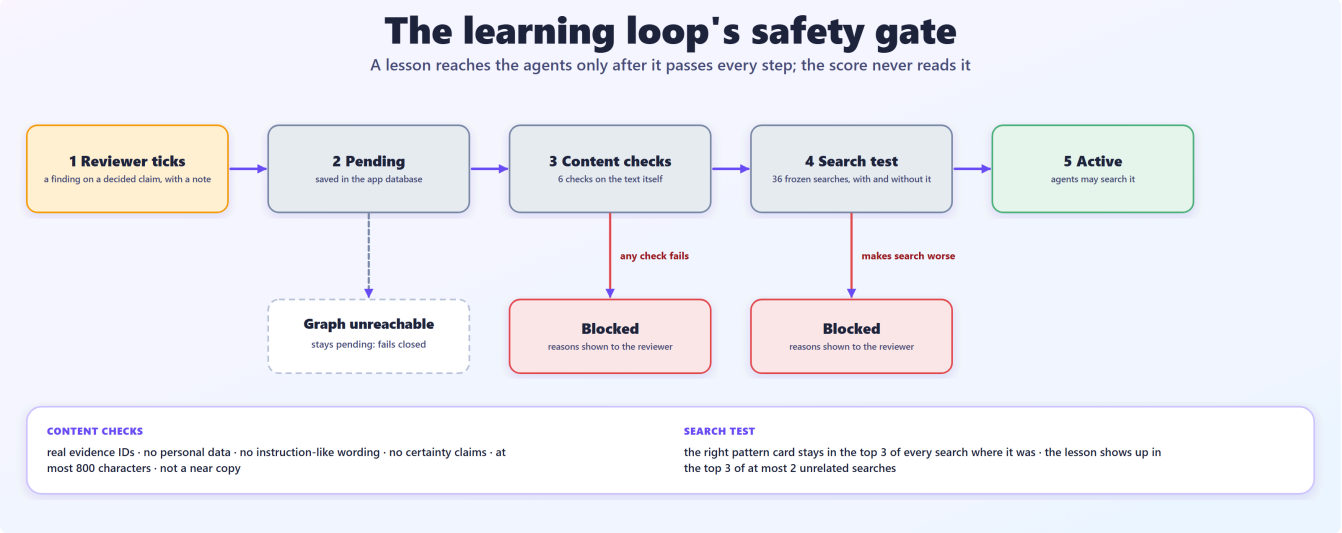
Navigation. The "Open" and "Back" buttons use a callback (`goto`) instead of changing state after the click. A callback runs before the script redraws, so it may set the sidebar radio's value, which Streamlit forbids after the radio is drawn.

Decision flow (`record_decision()`)

1. The reviewer name must not be empty.
2. The choice must be agree, override, hold or escalate.
3. For a band change: the new band must be valid, must differ from the system's band, and needs a note.
4. A hold needs a note saying what is missing. An escalation needs the name of another reviewer (not the person deciding) and a note saying why; it keeps the claim open and gives it to that person.
5. The final band is the system's band (agree, hold, escalate) or the reviewer's band (override). The decision is stored with the system's band and a flag that is true when they differ.
6. After saving, `next_unreviewed()` finds the next claim in queue order that has no decision other than a hold or an escalation, wrapping around to the start, and offers it.

Lesson flow

1. `candidate_signals()` takes the latest decision that is not a hold or an escalation per claim and lists that claim's findings, skipping the area and cover findings, each with its current status: new, pending, active or blocked.
2. `submit_signals()` validates every pick first (the claim is decided, the status is not pending or active, the note is long enough). Any problem stops the whole save and is shown in words.
3. Valid picks become `pending` rows in `signals`.
4. `run_safety_check()` calls the learning module. It moves each signal to active or blocked, or leaves it pending when the graph cannot be reached.



Plain-words layer. Reviewers never see a code such as `BILL_OUTLIER_VS_PEERS`. `finding_name()` takes the name from the pattern card (here "Inflated billing"), `finding_note()` the card's description and its list of innocent explanations, and `document_name()` a fixed table such as "Hospital bill" or "Police report (FIR)".

Empty and error states

Situation	What the page does
No stored assessments	An info box with the backfill command
No claim chosen	"Pick a claim in the review queue first"
No assessment for the claim	An error naming the claim
No saved agent run for the claim	An info line, then the rules-only result
An unknown evidence ID	A card titled "Not a record ID" or "(not found)", never an error

Situation	What the page does
Invalid decision or proposal	A red message that says what to fix; nothing is saved
Graph unreachable during the safety check	An amber "Still pending" banner; nothing is added
No result files in `eval/`	An info box asking to run the harness

10. Data, privacy, testing, configuration and limits

The dashboard reads only pseudonymous data, writes only its own database, and touches the graph through exactly one path: the safety check. The tables below make that boundary explicit.

What it reads and writes

Store	Access	Used for
`silver.duckdb`	read-only	claims, the reviewer view of customers, documents, providers, events, policies
`app.duckdb`	read and write	`assessments`, `decisions`, `signals`
`eval/*_cases.csv`	read-only	the Evaluation page
`knowledge/fraud_patterns.yaml`	read-only	plain finding names and descriptions
Neo4j graph	safety check only	reads the knowledge search; writes `ApprovedSignal` nodes
`vault.duckdb`, `labels.duckdb`	never opened	identity and answer keys

The app database

Table	Columns that matter
`assessments`	`claim_id`, `system` (`rules` or `agents`), `band`, `trust_score`, `evidence_status`, `result_json`, `source`, `created_at`; one row per claim and system

Table	Columns that matter
`decisions`	`decision_id`, `claim_id`, `reviewer`, `decision` (agree, override, hold or escalate), `final_band`, `note`, `system_band`, `disagrees_with_system`, `created_at`, `tags`, `escalated_to`
`assignments`	`claim_id`, `assignee`, `assigned_by`, `created_at`; who owns a claim now (set by hand or by an escalation); one row per claim
`signals`	`signal_id`, `source_claim_id`, `indicator_code`, `plain_text`, `evidence_ids`, `reviewer`, `note`, `status`, `gate_result`, `disagreed`, timestamps

Privacy on screen. Customers are shown by ID with coarse bands only: age band, gender, occupation, income band, location at district or state level (or "withheld" for small groups) and tenure. Record cards show scrubbed text. The sidebar says so on every page.

Testing. 53 of the project's 206 tests cover the app, in `tests/app`. They check the store and audit trail, the queue and its filters, decision validation and the next-claim rule, the lesson rules and the Evaluation metrics. Pages are also run headlessly with Streamlit's `AppTest`, which renders each page and clicks its widgets, and one test checks that the Evaluation page's source never mentions the answer-key database. The screenshots in this document were taken in a real browser on a scratch database.

Configuration

Setting	Effect
`CLAIM_TRIAGE_APP_DB`	Path of the app database. Used for demos, tests and the screenshots here
`--server.port`	The port Streamlit listens on; the preview configuration uses 8501
Neo4j credentials (environment or git-ignored file)	Needed only when the safety check runs
`config/data.yaml`, `config/weights.yaml`	Read indirectly: the data layout and the score that the assessments contain

Known limits

- **No login or access control.** The reviewer name is free text, so the audit trail records what was typed, not a verified identity. A hosted copy must be private.

- **One writer at a time.** DuckDB allows a single writing process, so the dashboard suits one reviewer; many simultaneous reviewers would need a server database.
- **A saved agent run is a stored copy.** It shows what was saved at the time, and the Evidence tile reads "Not recorded" for it.
- **Agent runs cover test claims only.** Saved agent runs exist for the 380 evaluated claims; a newly filed claim has only the rules-only assessment until a run is saved.
- **The lesson list shows the newest 50 findings.** More are offered as older ones are handled.
- **The queue table is drawn on a canvas.** Row selection is by mouse; automated tests and screenshots must click at coordinates instead of reading page elements.
- **Safety check depends on Neo4j.** The free Aura instance pauses when idle; the page then reports the lesson as pending until it is resumed.
- **Deployment.** A hosted copy needs the data built at start-up and a private app, because the disk may reset and decisions would be lost.

11. How the dashboard fits human-in-the-loop design

The dashboard is the checkpoint of an agentic system whose tools are read-only, so every claim's decision happens here. The agentic-systems course describes what a good review queue shows and which review actions to offer. This section compares the screens with that guidance and lists the gaps.

The Claim page against the course's review-queue anatomy

Course element	On the Claim page (section 5)
Task	the header: claim ID, customer ID, line, filing date and amount (component 2)
Recommended action	"Suggested next step" in "What was found" (component 6)
Reasoning summary	"Why this score" (component 5)
Evidence	each finding's evidence IDs, and "Look at a record" (component 7)
Tool calls	the list of tools the Investigator used, in the saved agent view
Confidence	the trust score and its bar (components 3 and 4) and each finding's confidence

The decision form against the course's review patterns

Course pattern	Dashboard
Approve	"Agree with the system's band"
Override	"Change the band", with a required reason
Hold for information	"Hold: more information needed", with a required note
Annotate	a note on every decision, an automatic flag when the reviewer disagrees with the system, and optional tags saying what was wrong with the output
Reject, edit, retry	not offered as separate buttons: changing the band covers a rejection, and the reasons cannot be edited
Escalate	"Escalate to someone else": names the other reviewer, needs a reason, keeps the claim open and gives it to that person. **Now built.**

The course's anti-patterns

- **Review everything.** Every claim passes a person, by design at this autonomy level. The queue ranks claims least trusted first so attention goes where the risk is.
- **Hide source evidence.** Avoided: every ID cited opens in the record viewer.
- **Alert fatigue.** The dashboard raises no alerts and uses three bands.
- **Vague approval prompts.** Avoided: four explicit choices, and a required reason for a change, a hold or an escalation.
- **Unclear ownership. Mostly closed.** Closed in part: a claim now has an owner (set by hand or by an escalation), shown on the claim page and in the queue. There is still no deadline.

Gaps and the small changes that close them (the first three are now built)

Gap	Change	Where in the code
Escalate (now built)	a fourth choice records who the claim goes to and why; the claim stays open and moves to that person	`claim_view.CHOICES`, `record_decision()` and the `decisions` table

Gap	Change	Where in the code
Structured tags (now built)	five optional tags (evidence did not support the finding, evidence missing, reasoning wrong, rule misread, wording off) saved with the decision, to build an evaluation set later	`decision_panel()` and the `decisions` table
Assignee (now built)	an "Assigned to" column and an "Assigned to me" filter in the queue, and an owner box on the claim page	`review_queue.py`, `store.py`
Tool calls are only a caption	show them as an expandable list with each result's size	`findings_section()`

None of these is built yet. Each needs a code change and tests, and the first three also touch the app database.